

ROB521: Mobile Robotics and Perception

Assignment #2: Wheel Odometry and Mapping

Student: Azure Lin

Question 1: Noise-Free Wheel Odometry

The odometry trajectory closely tracks the ground truth, with a maximum position error of ~ 0.018 m and heading error bounded within ± 0.025 rad. This small residual arises from Euler discretisation of the unicycle model:

$$\begin{cases} \theta_{k+1} = \theta_k + \omega_k \Delta t \\ x_{k+1} = x_k + v_k \cos \theta_k \Delta t \\ y_{k+1} = y_k + v_k \sin \theta_k \Delta t \end{cases}$$

The position update uses θ_k (the heading at the start of the interval), but in reality θ changes continuously in the interval $[t_k, t_{k+1}]$. When ω is large, the true trajectory is an arc, while Euler approximates it as a straight segment at the stale heading. Hence, position error peaks during high- ω turning segments (220–260 s), which is confirmed by the correlation between heading error spikes and position error growth in the bottom two plots.

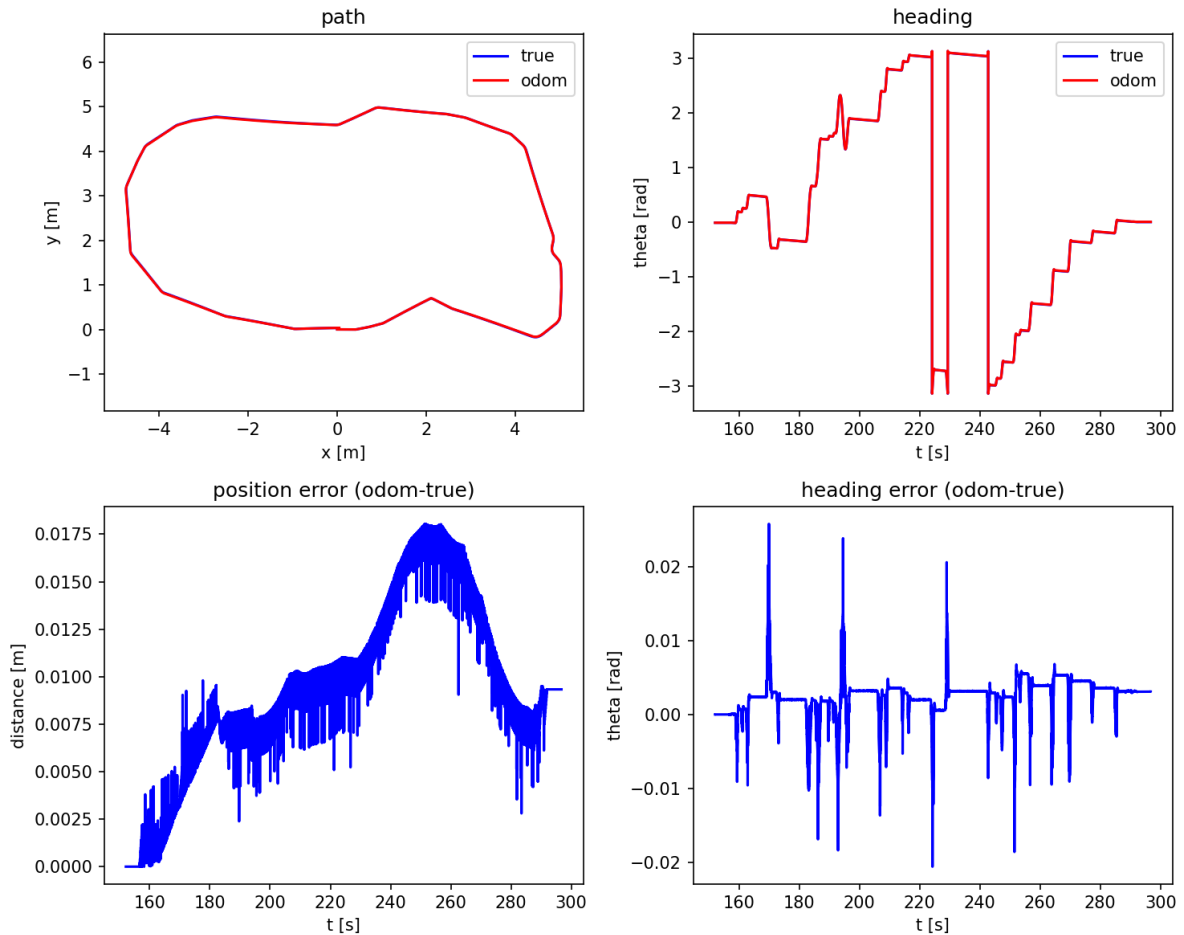


Figure 1: Noise-free wheel odometry vs. ground truth.

Question 2: Noisy Wheel Odometry

Adding Gaussian noise ($\sigma_v = 0.2$ m/s, $\sigma_\omega = 0.04$ rad/s) produces a fan-shaped spread of trajectories that widens over time. This is because angular velocity noise accumulates into heading error via a random walk

$$\theta_k = \theta_0 + \sum \omega_i \Delta t_i$$

and the corrupted heading then redirects all subsequent position updates through $\cos \theta_k$, $\sin \theta_k$. A small early heading error thus deflects the entire future trajectory, causing the unbounded drift characteristic of dead reckoning.

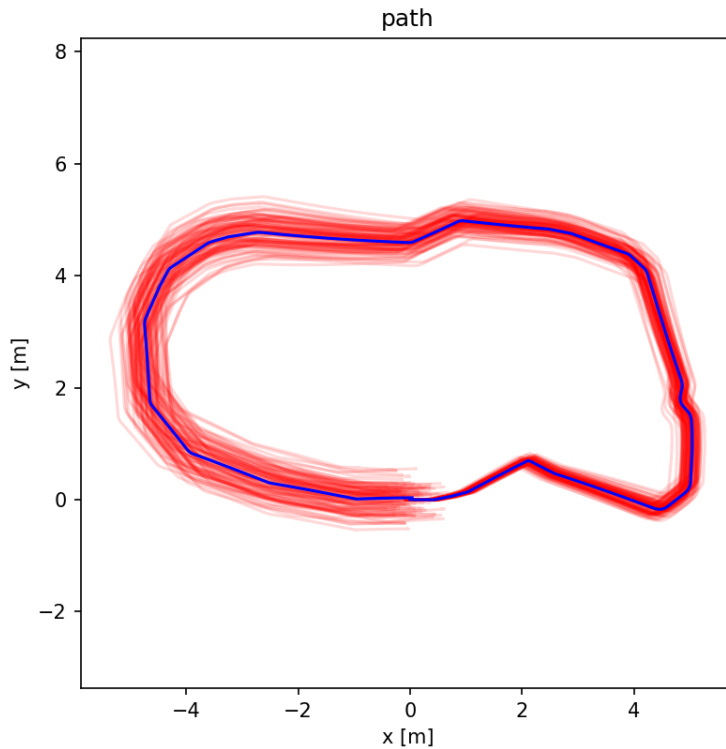


Figure 2: 100 noisy odometry trials (red) vs. ground truth (blue).

Question 3: Building a Map from Laser Scans

The ground truth map (blue) exhibits real walls. The noisy odometry map (red) shows blurred and shifted walls, with some surfaces appearing doubled. This doubling is particularly visible in the right-side area in the room. The robot drives through this region twice (once at the start and once after completing the loop), and while the early scan aligns well with the ground truth, the later scan is displaced by the drift accumulated over the full loop, producing two distinct red layers for the same wall. This degradation worsens for walls observed later in the trajectory, as odometry error is cumulative.

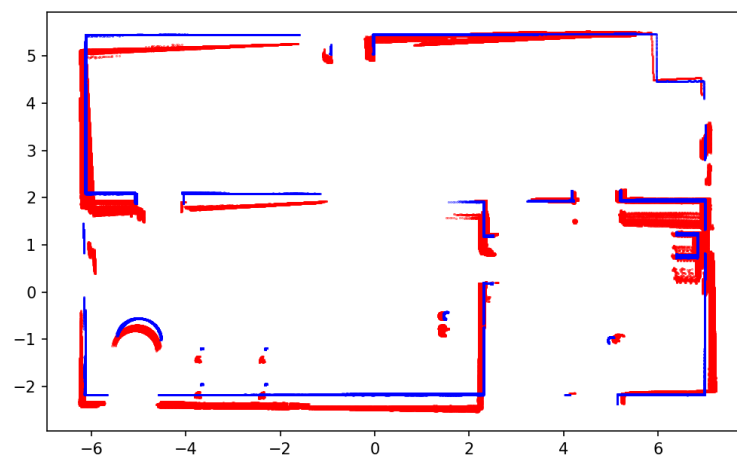


Figure 3: Laser scan map: ground truth poses (blue) vs. noisy odometry (red).

Appendix: Code

```
# =====
# ROB521_assignment2.py
# =====
#
# This assignment will introduce you to the idea of estimating the
# motion
# of a mobile robot using wheel odometry, and then also using that
# wheel
# odometry to make a simple map. It uses a dataset previously
# gathered in
# a mobile robot simulation environment called Gazebo. Watch the video
# ,
# 'gazebo.mp4' to visualize what the robot did, what its environment
# looks like, and what its sensor stream looks like.
#
# There are three questions to complete (5 marks each):
#
# Question 1: code (noise-free) wheel odometry algorithm
# Question 2: add noise to data and re-run wheel odometry algorithm
# Question 3: build a map from ground truth and noisy wheel
# odometry
#
# Fill in the required sections of this script with your code, run it
# to
# generate the requested plots, then paste the plots into a short
# report
# that includes a few comments about what you've observed. Append
# your
# version of this script to the report. Hand in the report as a PDF
# file.

import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat

np.random.seed(1)

# =====
# load the dataset from file
# =====
#
# ground truth poses: t_true x_true y_true theta_true
# laser scans: t_laser y_laser
# laser range limits: r_min_laser r_max_laser
# laser angle limits: phi_min_laser phi_max_laser

# load data (assumes the .mat file is in the same folder)
data = loadmat('ROB521_assignment2_gazebo_data.mat')

# helper to squeeze Matlab arrays to 1D where appropriate
squeeze = lambda a: np.array(a).squeeze()

t_true = squeeze(data['t_true'])
x_true = squeeze(data['x_true'])
y_true = squeeze(data['y_true'])
theta_true = squeeze(data['theta_true'])
```

```

t_odom = squeeze(data['t_odom'])
v_odom = squeeze(data['v_odom'])
omega_odom = squeeze(data['omega_odom'])

# laser
t_laser = squeeze(data['t_laser'])
y_laser = np.array(data['y_laser'])

r_min_laser = float(squeeze(data['r_min_laser']))
r_max_laser = float(squeeze(data['r_max_laser']))
phi_min_laser = float(squeeze(data['phi_min_laser']))
phi_max_laser = float(squeeze(data['phi_max_laser']))

# -----
# Question 1: noise-free wheel odometry
# -----

# Write an algorithm to estimate the pose of the robot throughout
# motion
# using the wheel odometry data (t_odom, v_odom, omega_odom) and
# assuming
# (x_odom y_odom theta_odom) so that the comparison plots can be
# generated
# below. See the plot 'ass2_q1_soln.png' for what your results should
# look
# like.

# variables to store wheel odometry pose estimates
numodom = t_odom.size
x_odom = np.zeros(numodom)
y_odom = np.zeros(numodom)
theta_odom = np.zeros(numodom)

# initialize from ground truth
x_odom[0] = x_true[0]
y_odom[0] = y_true[0]
theta_odom[0] = theta_true[0]

# -----insert your wheel odometry algorithm here-----
# get all time difference
dt = np.diff(t_odom)

# heading trajectory
theta_odom = np.empty(numodom) # shape: (numodom,)
theta_odom[0] = theta_true[0]
theta_odom[1:] = theta_true[0] + np.cumsum(omega_odom[:-1] * dt)

# build G(q) in differential kinematics bicycle model
G = np.zeros((numodom-1, 3, 2)) # shape: (numodom-1, 3, 2)
G[:,0,0] = np.cos(theta_odom[:-1]) # cos a 0
G[:,1,0] = np.sin(theta_odom[:-1]) # sin a 0
G[:,2,1] = 1.0 # 0 1

# stack all input velocities
p = np.column_stack( # shape: (numodom-1, 2)
    [v_odom[:-1], omega_odom[:-1]]
)

# compute all change in pose
dq = np.einsum('nij,nj->ni', G, p) * dt[:, None] # shape: (numodom-1,
3)

```

```

# construct the trajectory
q0 = np.array([x_true[0], y_true[0], theta_true[0]])
q = np.empty((numodom, 3))
q[0] = q0
q[1:] = q0 + np.cumsum(dq, axis=0)

# extract the three pose arrays
x_odom = q[:, 0]
y_odom = q[:, 1]
theta_odom = q[:, 2]
# -----end of your wheel odometry algorithm-----

# plots
fig, axs = plt.subplots(2,2, figsize=(10,8))
axs = axs.flatten()
axs[0].plot(x_true, y_true, 'b')
axs[0].plot(x_odom, y_odom, 'r')
axs[0].legend(['true','odom'])
axs[0].set_xlabel('x[m]')
axs[0].set_ylabel('y[m]')
axs[0].set_title('path')
axs[0].axis('equal')

# wrap heading to [-pi,pi]
theta_odom = np.arctan2(np.sin(theta_odom), np.cos(theta_odom))
axs[1].plot(t_true, theta_true, 'b')
axs[1].plot(t_odom, theta_odom, 'r')
axs[1].legend(['true','odom'])
axs[1].set_xlabel('t[s]')
axs[1].set_ylabel('theta[rad]')
axs[1].set_title('heading')

pos_err = np.sqrt((x_odom - x_true)**2 + (y_odom - y_true)**2)
axs[2].plot(t_odom, pos_err, 'b')
axs[2].set_xlabel('t[s]')
axs[2].set_ylabel('distance[m]')
axs[2].set_title('position_error(odom-true)')

# wrap angular error to [-pi,pi]
theta_err = np.arctan2(np.sin(theta_odom - theta_true), np.cos(
    theta_odom - theta_true))
axs[3].plot(t_odom, theta_err, 'b')
axs[3].set_xlabel('t[s]')
axs[3].set_ylabel('theta[rad]')
axs[3].set_title('heading_error(odom-true)')

plt.tight_layout()
plt.show(block=False)
fig.savefig('ass2_q1.png', dpi=150)

# =====
# Question 2: add noise to data and re-run wheel odometry algorithm
# =====
#
# Now we're going to deliberately add some noise to the linear and
# angular velocities to simulate what real wheel odometry is like.
# Copy
# your wheel odometry algorithm from above into the indicated place
# below

```

```

# to see what this does. The below loops 100 times with different
    random
# noise. See the plot 'ass2_q2_soln.png' for what your results should
    look
# like.

# save the original odometry variables for later use
v_odom_noisefree = np.copy(v_odom)
omega_odom_noisefree = np.copy(omega_odom)

fig2 = plt.figure(figsize=(6,6))
ax = fig2.add_subplot(1,1,1)

# loop multiple random trials
for n in range(100):

    # add noise to wheel odometry measurements (yes, on purpose to see
        effect)
    v_odom = v_odom_noisefree + 0.2 * np.random.randn(numodom)
    omega_odom = omega_odom_noisefree + 0.04 * np.random.randn(numodom
        )

    # -----insert your wheel odometry algorithm here-----
    # get all time difference
    dt = np.diff(t_odom)

    # heading trajectory
    theta_odom = np.empty(numodom)                # shape: (numodom,)
    theta_odom[0] = theta_true[0]
    theta_odom[1:] = theta_true[0] + np.cumsum(omega_odom[:-1] * dt)

    # build G(q) in differential kinematics bicycle model
    G = np.zeros((numodom-1, 3, 2))                # shape: (numodom-1, 3, 2)
    G[:,0,0] = np.cos(theta_odom[:-1])              # cos a    0
    G[:,1,0] = np.sin(theta_odom[:-1])              # sin a    0
    G[:,2,1] = 1.0                                  # 0        1

    # stack all input velocities
    p = np.column_stack(                            # shape: (numodom-1, 2)
        [v_odom[:-1], omega_odom[:-1]]
    )

    # compute all change in pose
    dq = np.einsum('nij,nj->ni', G, p) * dt[:, None] # shape: (
        numodom-1, 3)

    # construct the trajectory
    q0 = np.array([x_true[0], y_true[0], theta_true[0]])
    q = np.empty((numodom, 3))
    q[0] = q0
    q[1:] = q0 + np.cumsum(dq, axis=0)

    # extract the three pose arrays
    x_odom = q[:, 0]
    y_odom = q[:, 1]
    theta_odom = q[:, 2]
    # -----end of your wheel odometry algorithm-----

    ax.plot(x_odom, y_odom, color='r', alpha=0.15)

# plot ground truth on top

```

```

ax.plot(x_true, y_true, 'b')
ax.set_xlabel('x[m]')
ax.set_ylabel('y[m]')
ax.set_title('path')
ax.axis('equal')

plt.show(block=False)
fig2.savefig('ass2_q2.png', dpi=150)

# After this loop, v_odom and omega_odom remain as the last noisy
# trial and can be used in the next question
# to build a map from noisy odometry. You can also re-run this loop
# with different noise levels to see how
# it affects the results.

# -----
# Question 3: build map from laser scans using noisy and noise-free
# poses
# -----
# Now we're going to try to plot all the points from our laser scans
# in the
# robot's initial reference frame. This will involve first figuring
# out
# how to plot the points in the current frame, then transforming them
# back
# to the initial frame and plotting them. Do this for both the ground
# truth pose (blue) and also the last noisy odometry that you
# calculated in
# Question 2 (red). At first even the map based on the ground truth
# may
# not look too good. This is because the laser timestamps and
# odometry
# timestamps do not line up perfectly and you'll need to interpolate.
# Even
# after this, two additional patches will make your map based on
# ground
# truth look as crisp as the one in 'ass2_q3_soln.png'. The first
# patch is
# to only plot the laser scans if the angular velocity is less than
# 0.1 rad/s; this is because the timestamp interpolation errors have
# more
# of an effect when the robot is turning quickly. The second patch is
# to
# account for the fact that the origin of the laser scans is about 10
# cm
# behind the origin of the robot. Once your ground truth map looks
# crisp,
# compare it to the one based on the odometry poses, which should be
# far
# less crisp, even with the two patches applied.

fig3 = plt.figure(figsize=(8,8))
ax3 = fig3.add_subplot(1,1,1)

npoints = y_laser.shape[1]
angles = np.linspace(phi_min_laser, phi_max_laser, npoints)

for n in [1,2]:
    if n == 1:
        # noisy odometry at laser timestamps (use last noisy odometry
        # arrays)
        x_interp = np.interp(t_laser, t_odom, x_odom)

```



```

y_interp = np.interp(t_laser, t_odom, y_odom)
# unwrap theta before interpolation to avoid jumps
#theta_unwrapped = np.unwrap(theta_odom)
theta_interp = np.interp(t_laser, t_odom, theta_odom)
omega_interp = np.interp(t_laser, t_odom, omega_odom)
color = 'r'
else:
    # ground truth interpolation at laser timestamps
    x_interp = np.interp(t_laser, t_true, x_true)
    y_interp = np.interp(t_laser, t_true, y_true)
    #theta_unwrapped = np.unwrap(theta_true)
    theta_interp = np.interp(t_laser, t_true, theta_true)
    omega_interp = np.interp(t_laser, t_true, omega_odom_noisefree
        )
    color = 'b'

# loop over laser scans
for i in range(t_laser.size):
    # -----insert your point transformation algorithm here-----

    # step 0: skip scans during fast rotation
    if abs(omega_interp[i]) > 0.1:
        continue

    # step 1: filter out points that are out of range
    valid_mask = (y_laser[i] > r_min_laser) & (y_laser[i] <
        r_max_laser)
    valid_ranges = y_laser[i][valid_mask]
    valid_angles = angles[valid_mask]

    # step 2: convert polar coordinates to Cartesian coordinates
    # in the laser frame
    x_laser_frame = valid_ranges * np.cos(valid_angles)
    y_laser_frame = valid_ranges * np.sin(valid_angles)

    # step 3: transform points from laser frame to robot frame (
    # accounting for laser offset)
    x_robot_frame = x_laser_frame - 0.10
    y_robot_frame = y_laser_frame

    # step 4: transform points from robot frame to world frame
    # using the interpolated pose
    cos_theta = np.cos(theta_interp[i])
    sin_theta = np.sin(theta_interp[i])
    x_world = x_interp[i] + cos_theta * x_robot_frame - sin_theta
        * y_robot_frame
    y_world = y_interp[i] + sin_theta * x_robot_frame + cos_theta
        * y_robot_frame

    # step 5: plot the points in the world frame
    ax3.plot(x_world, y_world, '.', color=color, markersize=0.5)

    # -----end of your point transformation algorithm-----

ax3.set_aspect('equal')
plt.show()

# save each figure to disk
fig3.savefig('ass2_q3.png', dpi=150)

print('Saved ass2_q1.png, ass2_q2.png, ass2_q3.png')

```